



EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA



TECNM
TECNOLOGICO NACIONAL D
MÉXICO



INSTITUTO TECNOLÓGICO DE MÉXICO
CAMPUS ORIZABA

“ACTIVIDAD 6”

EQUIPO 9

GARCIA CEDEÑO IRVING EMANUEL

ROJAS BONILLA SANTIAGO

MARTINEZ ROSAS JOSE ANGEL

MATERIA: PROGRAMACION ORIENTADA A OBJETOS

GRUPO: VESPERTINO 1-2 PM

MAESTRO: RAFAEL HERRERA GARCIA

FECHA DE ENTREGA: 6 DE MAYO DE 2026

Problema 1:

En cierta empresa de fabricación de ropa, se requiere sistematizar el proceso, se fabrican diversidad de prendas de vestir tanto para caballeros como damas, no se fabrica ropa de niños, de cada prenda se tiene un registro de modelo, tela, costo de producción por pieza, género (masculino, femenino o mixto), temporada (primavera, verano, otoño o invierno), cuando se produce un lote de cada prenda, se registra número de lote, número de piezas, fecha de fabricación. De cada lote debe calcularse el costo de producción del lote, así como el monto de recuperación que se obtendrá después de su venta, el precio de venta por pieza es siempre 15% más del costo de producción y por lote completo es por cada pieza el 5% más del costo de producción.

El programa que se desarrolle deberá gestionar una colección de prendas y una colección de lotes. El programa deberá validar el género (Masculino, femenino o mixto), la temporada (primavera, verano, otoño o invierno), Los costos de producción de una prenda nunca deben rebasar un límite impuesto cuando una prenda se crea. El número de piezas en cada lote debe ser entre 50 y 350.

Análisis:

Validación de género: solo permite Masculino, femenino y mixto, si no son uno de estos el programa lanzara una excepción.

Validación de temporada: solo se permiten, primavera, verano, otoño y invierno, si no son una de estas, el programa lanzara una excepción.

Validación costo de producción de una prenda: no puede rebasar el límite de impuesto cuando la prenda se crea, si no es así el programa lanzara una excepción.

Validación de número de piezas por lote: el lote puede tener entre 50 y 350 piezas cada uno, si no es así el programa lanzara una excepción.

Código clase prenda:

```
© Lote.java  © Prenda.java x
1 package Fabrica.Ropa;
2
3 import java.util.Objects;
4
5 public class Prenda implements Comparable<Prenda> { 5 usages  Irving Emanuel Garcia Cedeño *
6     private String Modelo; 7 usages
7     private String Tela; 4 usages
8     private double PrecioVentaPZ; 4 usages
9     private double PrecioVentaLote; 2 usages
10    private String Genero; 4 usages
11    private String Temporada; 4 usages
12
13    @ public Prenda(String modelo, String tela, double precioVentaPZ, double precioVentaLote, String genero, String temporada) {
14        if (!(genero.equalsIgnoreCase( anotherString: "Masculino") ||
15            genero.equalsIgnoreCase( anotherString: "Femenino") ||
16            genero.equalsIgnoreCase( anotherString: "Mixto"))) {
17            throw new IllegalArgumentException("Genero inválido");
18        }
19
20        if (!(temporada.equalsIgnoreCase( anotherString: "Primavera") ||
21            temporada.equalsIgnoreCase( anotherString: "Verano") ||
22            temporada.equalsIgnoreCase( anotherString: "Otoño") ||
23            temporada.equalsIgnoreCase( anotherString: "Invierno"))) {
24            throw new IllegalArgumentException("Temporada inválida");
25        }
26    }
```

```
© Lote.java x  © Prenda.java x
5     public class Prenda implements Comparable<Prenda> { 5 usages  Irving Emanuel Garcia Cedeño *
13     public Prenda(String modelo, String tela, double precioVentaPZ, double precioVentaLote, String genero, String temporada) {
27         if (precioVentaPZ < 0 || precioVentaLote < 0) {
28             throw new IllegalArgumentException("Precios negativos no permitidos");
29         }
30
31         this.Modelo = modelo;
32         this.Tela = tela;
33         this.PrecioVentaPZ = precioVentaPZ;
34         this.PrecioVentaLote = precioVentaLote;
35         this.Genero = genero;
36         this.Temporada = temporada;
37     }
38
39     public String getModelo() { no usages  Irving Emanuel Garcia Cedeño
40         return Modelo;
41     }
42
43     public void setModelo(String modelo) { no usages  Irving Emanuel Garcia Cedeño
44         Modelo = modelo;
45     }
46
47     public String getTela() { no usages  Irving Emanuel Garcia Cedeño
48         return Tela;
49     }
50 }
```

```

Lote.java  Prenda.java x
5      public class Prenda implements Comparable<Prenda> { 5
51          public void setTela(String tela) { no usages  ⓘ
52              Tela = tela;
53          }
54
55          public String getGenero() { no usages  ⓘ Irving Ema
56              return Genero;
57          }
58
59          public void setGenero(String genero) { no usage
60              Genero = genero;
61          }
62
63          public String getTemporada() { no usages  ⓘ Irving
64              return Temporada;
65          }
66
67          public void setTemporada(String temporada) { n
68              Temporada = temporada;
69          }
70
71          public double precioVentaPZ() { 2 usages  ⓘ Irving
72              return PrecioVentaPZ * 1.15;
73          }

```

```

Lote.java  Prenda.java x
5      public class Prenda implements Comparable<Prenda> { 5 usages  ⓘ Irving Emanuel Garc
75          public double precioventaLote() { no usages  ⓘ Irving Emanuel Garcia Cedeño
76              return PrecioVentaPZ * 1.05;
77          }
78          @Override  ⓘ Irving Emanuel Garcia Cedeño
79          public int compareTo(Prenda p) {
80              return Double.compare(this.precioVentaPZ(), p.precioVentaPZ());
81          }
82          @Override  ⓘ Irving Emanuel Garcia Cedeño
83          public boolean equals(Object obj) {
84              if (this == obj) return true;
85              if (!(obj instanceof Prenda)) return false;
86              Prenda p = (Prenda) obj;
87              return Modelo.equals(p.Modelo);
88          }
89          @Override  ⓘ Irving Emanuel Garcia Cedeño
90          public int hashCode() {
91              return Objects.hash(Modelo);
92          }

```

```

@Override  Irving Emanuel Garcia Cedeño
public String toString() {
    return "prenda{" +
        "modelo='" + Modelo + '\'' +
        ", Tela='" + Tela + '\'' +
        ", precioventaPZ=" + PrecioVentaPZ +
        ", precioVentaLote=" + PrecioVentaLote +
        ", genero='" + Genero + '\'' +
        ", temporada='" + Temporada + '\'' +
        '}';
}
}

```

Código clase lote:

```

Lote.java x Prenda.java
1 package Fabrica.Ropa;
2
3 import java.time.LocalDate;
4
5 public class Lote { no usages Irving Emanuel Garcia Cedeño
6     private int NumDeLote; 4 usages
7     private int NumDePiezas; 6 usages
8     private LocalDate FechadeFabricacion; 4 usages
9     private double CostoProduccionPz; 6 usages
10
11     public Lote(int numDeLote, int numDePiezas, LocalDate fechadeFabricacion, double costoProduccionPz) {
12
13         if (numDePiezas < 50 || numDePiezas > 350) {
14             throw new IllegalArgumentException("Piezas deben estar entre 50 y 350");
15         }
16
17         if (costoProduccionPz < 0) {
18             throw new IllegalArgumentException("Costo inválido");
19         }
20
21         this.NumDeLote = numDeLote;
22         this.NumDePiezas = numDePiezas;
23         this.FechadeFabricacion = fechadeFabricacion;
24         this.CostoProduccionPz = costoProduccionPz;
25     }

```



```

Lote.java x Prenda.java
5 public class Lote { no usages Irving Emanuel Garcia Cedeño
6
7     public int getNumDeLote() { no usages Irving Emanuel Garcia Cedeño
8         return NumDeLote;
9     }
10
11     public void setNumDeLote(int numDeLote) { no usages Irving Emanuel Garcia Cedeño
12         NumDeLote = numDeLote;
13     }
14
15     public int getNumDePiezas() { no usages Irving Emanuel Garcia Cedeño
16         return NumDePiezas;
17     }
18
19     public void setNumDePiezas(int numDePiezas) { no usages Irving Emanuel Garcia Cedeño
20         NumDePiezas = numDePiezas;
21     }
22
23     public LocalDate getFechaDeFabricacion() { no usages Irving Emanuel Garcia Cedeño
24         return FechaDeFabricacion;
25     }
26
27     public void setFechaDeFabricacion(LocalDate fechaDeFabricacion) {
28         FechaDeFabricacion = fechaDeFabricacion;
29     }
30
31     public double getCostoProduccionPz() { no usages Irving Emanuel Garcia Cedeño
32         return CostoProduccionPz;
33     }

```

```

Lote.java x Prenda.java
5 public class Lote { no usages Irving Emanuel Garcia Cedeño
6
7     public void setCostoProduccionPz(double costoProduccionPz) { no usages Irving Emanuel Garcia Cedeño
8         CostoProduccionPz = costoProduccionPz;
9     }
10
11     public double costoProduccionLote(){ no usages Irving Emanuel Garcia Cedeño
12         return NumDePiezas*CostoProduccionPz;
13     }
14     public double calcularMontoRecuperacion(){ no usages Irving Emanuel Garcia Cedeño
15
16         if (CostoProduccionPz == 0) {
17             throw new ArithmeticException("Costo no puede ser 0");
18         }
19
20         double precioVenta=CostoProduccionPz*1.15;
21         return NumDePiezas*precioVenta;
22     }
23
24     @Override Irving Emanuel Garcia Cedeño
25     public String toString() {
26         return "lote{" +
27             "numDeLote=" + NumDeLote +
28             ", numDePiezas=" + NumDePiezas +
29             ", fechaDeFabricacion=" + FechaDeFabricacion +
30             '}';
31     }
32 }

```

Prueba:

```
jshell> import Fabrica.Ropa.*;

jshell> new Prenda("Playera", "Algodón", 100, 90, "Otro", "Verano");
| Exception java.lang.IllegalArgumentException: Genero inválido
|   at Prenda.<init> (Prenda.java:17)
|   at (#5:1)

jshell> new Prenda("Playera", "Algodón", 100, 90, "Masculino", "Navidad");
| Exception java.lang.IllegalArgumentException: Temporada inválida
|   at Prenda.<init> (Prenda.java:24)
|   at (#6:1)

jshell> new Prenda("Playera", "Algodón", -10, 90, "Masculino", "Verano");
| Exception java.lang.IllegalArgumentException: Precios negativos no permitidos
|   at Prenda.<init> (Prenda.java:28)
|   at (#7:1)

jshell> Prenda p1 = new Prenda("Playera", "Algodón", 100, 90, "Masculino", "Verano");
p1 ==> prenda{modelo='Playera', Tela='Algodón', preciove ... lino', temporada='Verano'}
```

```
jshell> import java.time.LocalDate;

jshell> new Lote(1, 10, LocalDate.now(), 50);
| Exception java.lang.IllegalArgumentException: Piezas deben estar entre 50 y 350
|   at Lote.<init> (Lote.java:14)
|   at (#10:1)

jshell> new Lote(1, 100, LocalDate.now(), -20);
| Exception java.lang.IllegalArgumentException: Costo inválido
|   at Lote.<init> (Lote.java:18)
|   at (#11:1)

jshell> Lote l1 = new Lote(1, 100, LocalDate.now(), 0);
l1 ==> lote{numDeLote=1, numDePiezas=100, fechadeFabricacion=2026-05-03}

jshell> l1.calcularMontoRecuperacion();
| Exception java.lang.ArithmeticException: Costo no puede ser 0
|   at Lote.calcularMontoRecuperacion (Lote.java:65)
|   at (#13:1)

jshell> Lote l2 = new Lote(2, 100, LocalDate.now(), 50);
l2 ==> lote{numDeLote=2, numDePiezas=100, fechadeFabricacion=2026-05-03}
```

Problema 2:

Una empresa agrícola ha decidido sistematizar el control de la producción frutal que tiene, para lo cual de cada fruta se describe su nombre, la extensión del terreno (hectáreas) dedicadas a su cultivo y producción, el tiempo de cosecha (Cada fruta puede tener 1 o más periodos de cosecha), la cantidad estimada a cosechar por hectárea en cada periodo (cada periodo por condiciones de estación climática puede tener diferentes cantidades estimadas), costo de producción promedio por tonelada y el precio de venta promedio por tonelada. Por cada fruta se puede agregar o eliminar un periodo de cosecha, se puede obtener el costo total de un periodo, así como las ganancias estimadas en un periodo.

EL programa que se desarrolle deberá gestionar colección de frutas y colección de periodos, el tiempo de cosecha debe ser un valor de 1 a 6, la extensión de terreno debe ser un valor de 0.5 hasta un límite impuesto al momento de crear un objeto de la clase Fruta, El cual nunca debe ser mayor a 10, la cantidad estimada de cosecha debe ser un valor entre 0.5 y hasta 200. El precio promedio debe ser siempre superior al costo promedio de producción. Entre el 5 y no más del 30% del costo promedio.

Análisis:

Validación del tiempo de cosecha: debe ser un valor de 1 a 6, si no es así el programa lanzara una excepción.

Validación de extensión del terreno: la extensión de hectáreas debe ser mínimo de 0.5 hectáreas, hasta un límite máximo definido al crear el objeto, ese límite máximo no puede ser mayor a 10 hectáreas, si no es así el programa lanzara una excepción.

Validación de cantidad estimada de cosecha: la producción estimada por hectárea debe ser de estar entre 0.5 y 200, si no es así el programa la rechazara.

Validación relación entre costo y precio venta: el precio promedio por tonelada debe ser siempre mayor al costo promedio de producción, debe estar entre 5% y 30% por encima del costo, no se permite un precio menor o igual al costo, ni que el precio supere el 30% del costo, si no es así el programa lanzara una excepción.

Código Clase Fruta:

```
Fruta.java x PeriodoCosecha.java
1 package empresa.agricola;
2
3 import java.util.ArrayList;
4 import java.util.Objects;
5
6 public class Fruta implements Comparable<Fruta> { 5 usages  Irving Emanuel Garcia Cedeño *
7     private String NombreFruta; 9 usages
8     private double Hectareas; 4 usages
9     private double CostoProduccionTon; 4 usages
10    private double PrecioVentaTon; 4 usages
11    private double LimiteHectareas; 1 usage
12    private ArrayList<PeriodoCosecha> Periodos; 5 usages
13
14    public Fruta(String nombreFruta, double hectareas, double costoProduccionTon, no usages  Irving Emanuel Garcia Cedeño *
15        ArrayList<PeriodoCosecha> periodos, double precioVentaTon, double limiteHectareas) {
16        if (limiteHectareas > 10) {
17            throw new IllegalArgumentException("El límite de hectáreas no puede ser mayor a 10");
18        }
19
20        if (hectareas < 0.5 || hectareas > limiteHectareas) {
21            throw new IllegalArgumentException("Hectáreas fuera de rango");
22        }
23
24        if (costoProduccionTon < 0) {
25            throw new IllegalArgumentException("Costo inválido");
26        }
27
```

```
Fruta.java x PeriodoCosecha.java
6 public class Fruta implements Comparable<Fruta> { 5 usages  Irving Emanuel Garcia Cedeño *
14 public Fruta(String nombreFruta, double hectareas, double costoProduccionTon, no usages  Irving Emanuel Garcia Cedeño *
28     if (precioVentaTon <= costoProduccionTon) {
29         throw new IllegalArgumentException("El precio debe ser mayor al costo");
30     }
31
32     double margen = (precioVentaTon - costoProduccionTon) / costoProduccionTon * 100;
33
34     if (margen < 5 || margen > 30) {
35         throw new IllegalArgumentException("El margen debe estar entre 5% y 30%");
36     }
37
38     this.NombreFruta = nombreFruta;
39     this.Hectareas = hectareas;
40     this.CostoProduccionTon = costoProduccionTon;
41     this.Periodos = periodos;
42     this.PrecioVentaTon = precioVentaTon;
43     this.LimiteHectareas = limiteHectareas;
44 }
45
46 public String getNombreFruta() { no usages  Irving Emanuel Garcia Cedeño *
47     return NombreFruta;
48 }
49
50 public void setNombreFruta(String nombreFruta) { no usages  Irving Emanuel Garcia Cedeño *
51     NombreFruta = nombreFruta;
52 }
```

```

Fruta.java x PeriodoCosecha.java
6      public class Fruta implements Comparable<Fruta> { 5 usages  Irving Emanuel Garcia Cedeño
54          public double getHectareas() { no usages  Irving Emanuel Garcia Cedeño
55              return Hectareas;
56          }
57
58          public void setHectareas(double hectareas) { no usages  Irving Emanuel Garcia Cedeño
59              Hectareas = hectareas;
60          }
61
62          public double getCostoProduccionTon() { no usages  Irving Emanuel Garcia Cedeño
63              return CostoProduccionTon;
64          }
65
66          public void setCostoProduccionTon(double costoProduccionTon) {
67              CostoProduccionTon = costoProduccionTon;
68          }
69
70          public double getPrecioVentaTon() { no usages  Irving Emanuel Garcia Cedeño
71              return PrecioVentaTon;
72          }
73
74          public void setPrecioVentaTon(double precioVentaTon) { no usages  Irving Emanuel Garcia Cedeño
75              PrecioVentaTon = precioVentaTon;
76          }

```

```

Fruta.java x PeriodoCosecha.java
6      public class Fruta implements Comparable<Fruta> { 5 usages  Irving Emanuel Garcia Cedeño *
78          public ArrayList<PeriodoCosecha> getPeriodos() { no usages  Irving Emanuel Garcia Cedeño
79              return Periodos;
80          }
81
82          public void agregarPeriodo(PeriodoCosecha p) { no usages  Irving Emanuel Garcia Cedeño
83              if (p == null) {
84                  throw new NullPointerException("El periodo de cosecha no puede ser nulo");
85              }
86              Periodos.add(p);
87          }
88
89          public void eliminarPeriodo(PeriodoCosecha periodo) { no usages  Irving Emanuel Garcia Cedeño
90              Periodos.remove(periodo);
91          }
92
93          public double totalProduccion() { no usages  Irving Emanuel Garcia Cedeño
94              double total = 0;
95              for (PeriodoCosecha p : Periodos) {
96                  total += p.getCantidadToneladas();
97              }
98              return total;
99          }

```

```

Fruta.java × PeríodoCosecha.java
6      public class Fruta implements Comparable<Fruta> { 5 usages  Irving
01          @Override  Irving Emanuel García Cedeño
02          public int compareTo(Fruta f) {
03              return this.NombreFruta.compareTo(f.NombreFruta);
04          }
05
06          @Override  Irving Emanuel García Cedeño
07          public boolean equals(Object obj) {
08              if (this == obj) return true;
09              if (!(obj instanceof Fruta)) return false;
10              Fruta f = (Fruta) obj;
11              return NombreFruta.equals(f.NombreFruta);
12          }
13
14          @Override  Irving Emanuel García Cedeño
15          public int hashCode() {
16              return Objects.hash(NombreFruta);
17          }
18
19          @Override  Irving Emanuel García Cedeño
20          public String toString() {
21              return "Fruta{" +
22                  "NombreFruta='" + NombreFruta + '\'' +
23                  ", Hectareas=" + Hectareas +
24                  ", CostoProduccionTon=" + CostoProduccionTon +
25                  ", PrecioVentaTon=" + PrecioVentaTon +
26                  '}';

```

Código Clase PeríodoCosecha

```

Fruta.java × PeríodoCosecha.java ×
1      package empresa.agricola;
2
3      public class PeríodoCosecha { 6 usages  Irving Emanuel García Cedeño *
4          private String NombrePeríodo; 4 usages
5          private double CantidadToneladas; 5 usages
6          private int TiempoCosecha; 4 usages
7
8          public PeríodoCosecha(String nombrePeríodo, double cantidadToneladas, int TiempoCosecha) {
9              if (TiempoCosecha < 1 || TiempoCosecha > 6) {
10                  throw new IllegalArgumentException("Tiempo de cosecha debe ser entre 1 y 6");
11              }
12
13              if (cantidadToneladas < 0.5 || cantidadToneladas > 200) {
14                  throw new IllegalArgumentException("Cantidad de toneladas fuera de rango");
15              }
16
17              this.NombrePeríodo = nombrePeríodo;
18              this.CantidadToneladas = cantidadToneladas;
19              this.TiempoCosecha = TiempoCosecha;
20          }
21
22          public String getNombrePeríodo() { no usages  Irving Emanuel García Cedeño
23              return NombrePeríodo;
24          }

```



```

© Fruta.java    © PeriodoCosecha.java x
3      public class PeriodoCosecha { 6 usages  Irving Emanuel Garcia Cedeño *
26      public void setNombrePeriodo(String nombrePeriodo) { no usages  Irving Emar
27          NombrePeriodo = nombrePeriodo;
28      }
29
30      public double getCantidadToneladas() { 1 usage  Irving Emanuel Garcia Cedeño
31          return CantidadToneladas;
32      }
33
34      public void setCantidadToneladas(double cantidadToneladas) { no usages  I
35          CantidadToneladas = cantidadToneladas;
36      }
37
38      public int getTiempoCosecha() { no usages  Irving Emanuel Garcia Cedeño
39          return TiempoCosecha;
40      }
41
42      public void setTiempoCosecha(int tiempoCosecha) { no usages  Irving Emanuel
43          TiempoCosecha = tiempoCosecha;
44      }
45
46      public double ProduccionTotal(double Hectareas) { 2 usages  Irving Emanuel G
47          return CantidadToneladas*Hectareas;
48      }
49      }
50      public double CostoProduccion(double Hectareas, double CostoTonleadadas){
51          return ProduccionTotal(Hectareas)*CostoTonleadadas;

```

```

public double gananciaTotal(double Hectareas, double CostoToneladas, double precioVenta){
    double produccion=ProduccionTotal(Hectareas);
    return (produccion*precioVenta)- (produccion*CostoToneladas);
}

```

```

@Override  Irving Emanuel Garcia Cedeño

```

```

public String toString() {
    return "PeriodoCosecha{" +
        "NombrePeriodo=" + NombrePeriodo + '\'' +
        ", CantidadToneladas=" + CantidadToneladas +
        ", TiempoCosecha=" + TiempoCosecha +
        '}';
}

```

```

}

```

Prueba:

```
jshell> ArrayList<PeriodoCosecha> lista = new ArrayList<>();
lista ==> []

jshell> new Fruta("Manzana", 5, 100, lista, 120, 15);
| Exception java.lang.IllegalArgumentException: El límite de hectáreas no puede ser mayor a 10
|   at Fruta.<init> (Fruta.java:16)
|   at (#30:1)

jshell> new Fruta("Manzana", 0.2, 100, lista, 120, 5);
| Exception java.lang.IllegalArgumentException: Hectáreas fuera de rango
|   at Fruta.<init> (Fruta.java:20)
|   at (#31:1)

jshell> new Fruta("Manzana", 5, 100, lista, 90, 5);
| Exception java.lang.IllegalArgumentException: El precio debe ser mayor al costo
|   at Fruta.<init> (Fruta.java:28)
|   at (#32:1)

jshell> new Fruta("Manzana", 5, 100, lista, 102, 5);
| Exception java.lang.IllegalArgumentException: El margen debe estar entre 5% y 30%
|   at Fruta.<init> (Fruta.java:34)
|   at (#33:1)

jshell> new Fruta("Manzana", 5, 100, lista, 200, 5);
| Exception java.lang.IllegalArgumentException: El margen debe estar entre 5% y 30%
|   at Fruta.<init> (Fruta.java:34)
|   at (#34:1)

jshell> Fruta f1 = new Fruta("Manzana", 5, 100, lista, 120, 5);
f1 ==> Fruta{NombreFruta='Manzana', Hectareas=5.0, Costo ... 0.0, PrecioVentaTon=120.0}
```

```
jshell> new PeriodoCosecha("Primera", 50, 10);
| Exception java.lang.IllegalArgumentException: Tiempo de cosecha debe ser entre 1 y 6
|   at PeriodoCosecha.<init> (PeriodoCosecha.java:14)
|   at (#36:1)

jshell> new PeriodoCosecha("Primera", 300, 3);
| Exception java.lang.IllegalArgumentException: Cantidad de toneladas fuera de rango
|   at PeriodoCosecha.<init> (PeriodoCosecha.java:18)
|   at (#37:1)

jshell> new PeriodoCosecha("Primera", 0.2, 3);
| Exception java.lang.IllegalArgumentException: Cantidad de toneladas fuera de rango
|   at PeriodoCosecha.<init> (PeriodoCosecha.java:18)
|   at (#38:1)

jshell> PeriodoCosecha pc1 = new PeriodoCosecha("Primera", 50, 3);
pc1 ==> PeriodoCosecha{NombrePeriodo='Primera', CantidadToneladas=50.0, TiempoCosecha=3}

jshell> lista.add(pc1);
$40 ==> true
```


Problema 3:

El observatorio astronómico de San Pedro Mártir, de acuerdo a sus observaciones necesita un catálogo de los cuerpos celestes descubiertos para lo cual de cada cuerpo celeste se describe la ubicación en el espacio estelar en relación a las posiciones geográficas espaciales (expresado en grado de latitud (norte o sur) y grados de longitud (este u oeste)) periodo de ubicación(mes(es)donde se observó en dicha posición), distancia aproximada a la tierra(años luz o km/segundos), el nombre que se le asigno, la composición del cuerpo estelar (gases, solidos, líquidos). De cada cuerpo celeste se puede obtener el desplazamiento con respecto a otra observación del mismo cuerpo en un periodo distinto.

El programa que se va a desarrollar debe considerar la gestión de una colección de cuerpos celestes, **deberá validarse la latitud y la longitud, los grados nunca deberán superar los 180, el periodo deberá validar que el mes sea válido.**

Análisis:

Validación de latitud: el valor número no puede pasar los 180 grados, no se permiten valores mayores a 180, si se ingresa un valor fuera del rango permitido, el programa lanzara una excepción.

Validación de longitud: el valor número no puede pasar los 180 grados, no se permiten valores fuera del rango, si se ingresa un valor fuera del rango permitido el programa lanzara una excepción.

Validación mes: el periodo debe representar un mes del año valido, solo se permiten de los meses de enero a diciembre (enero, febrero, marzo, abril, mayo, junio, julio, agosto, septiembre, octubre, noviembre, diciembre), en caso de no ser así el programa lanzara una excepción.

Código Clase CuerposCelestes:

```

1 package Observatorio;
2
3 import java.util.ArrayList;
4 import java.util.Objects;
5
6 public class CuerposCelestes implements Comparable<CuerposCelestes> { 5 usages  Irving Emanuel
7     private String Nombre; 7 usages
8     private String Composicion; 4 usages
9     private double DistanciaTierra; 4 usages
10    private String UnidadTierra; 4 usages
11    private ArrayList<PeriodoVisible> Periodos; 4 usages
12
13    public CuerposCelestes(String nombre, String composicion, double distanciaTierra, no
14        String unidadTierra, ArrayList<PeriodoVisible> periodos) {
15        Nombre = nombre;
16        Composicion = composicion;
17        DistanciaTierra = distanciaTierra;
18        UnidadTierra = unidadTierra;
19        Periodos = periodos;
20
21        if (distanciaTierra < 0) {
22            throw new IllegalArgumentException("Distancia inválida");
23        }
24
25        if (periodos == null) {
26            throw new NullPointerException("La lista de periodos no puede estar vacía");
27        }
28    }

```

```

6 public class CuerposCelestes implements Comparable<CuerposCelestes> {
30    public String getNombre() { no usages  Irving Emanuel Garcia Cedeño
31        return Nombre;
32    }
33
34    public void setNombre(String nombre) { no usages  Irving Emanuel Garcia Cedeño
35        Nombre = nombre;
36    }
37
38    public String getComposicion() { no usages  Irving Emanuel Garcia Cedeño
39        return Composicion;
40    }
41
42    public void setComposicion(String composicion) { no usages  Irving Emanuel Garcia Cedeño
43        Composicion = composicion;
44    }
45
46    public double getDistanciaTierra() { 2 usages  Irving Emanuel Garcia Cedeño
47        return DistanciaTierra;
48    }
49
50    public void setDistanciaTierra(double distanciaTierra) { no usages
51        DistanciaTierra = distanciaTierra;
52    }
53

```

```

CuerposCelestes.java x PerodoVisible.java UbicacionEspacial.java
6 public class CuerposCelestes implements Comparable<CuerposCelestes> { 5 usages Irving
54     public String getUnidadTierra() { no usages Irving Emanuel García Cedeño
55         return UnidadTierra;
56     }
57
58     public void setUnidadTierra(String unidadTierra) { no usages Irving Emanuel García Ced
59         UnidadTierra = unidadTierra;
60     }
61
62     public ArrayList<PeriodoVisible> getPeriodos() { no usages Irving Emanuel García Cedeño
63         return Periodos;
64     }
65
66     public void agregarPeriodo(PeriodoVisible p){ no usages Irving Emanuel García Cedeño
67         if (p == null) {
68             throw new NullPointerException("No se puede agregar un periodo vacio");
69         }
70         Periodos.add(p);
71     }
72     public void eliminarPeriodo(PeriodoVisible p){ no usages Irving Emanuel García Cedeño
73         Periodos.remove(p);
74     }
75     @Override Irving Emanuel García Cedeño
76     public int compareTo(CuerposCelestes c) {
77         return Double.compare(this.getDistanciaTierra(), c.getDistanciaTierra());
78     }

```

```

CuerposCelestes.java x PerodoVisible.java UbicacionEspacial.java
6 public class CuerposCelestes implements Comparable<CuerposCelestes> {
79     @Override Irving Emanuel García Cedeño
80     public boolean equals(Object obj) {
81         if (this == obj) return true;
82         if (!(obj instanceof CuerposCelestes)) return false;
83         CuerposCelestes c = (CuerposCelestes) obj;
84         return Nombre.equals(c.Nombre);
85     }
86     @Override Irving Emanuel García Cedeño
87     public int hashCode() {
88         return Objects.hash(Nombre);
89     }
90
91     @Override Irving Emanuel García Cedeño
92     public String toString() {
93         return "cuerposCelestes{" +
94             "Nombre='" + Nombre + '\'' +
95             ", Composicion='" + Composicion + '\'' +
96             ", DistanciaTierra=" + DistanciaTierra +
97             ", UnidadTierra='" + UnidadTierra + '\'' +
98             '}';
99     }
100 }

```

Código clase ubicacionEspacial:

```
© CuerposCelestes.java  © PeriodoVisible.java  © UbicacionEspacial.java ×
1  package Observatorio;
2
3  public class UbicacionEspacial { 5 usages  👤 Irving Emanuel García Cedeño
4      private double Latitud; 6 usages
5      private String HemisferioLatitud; 3 usages
6      private double Longitud; 6 usages
7      private String HemisferioLongitud; 3 usages
8
9      public UbicacionEspacial(double latitud, String hemisferioLatitud, String hemisferioLongitud, double longitud) {
10
11          if (latitud < -90 || latitud > 90) {
12              throw new IllegalArgumentException("Latitud fuera de rango");
13          }
14
15          if (longitud < -180 || longitud > 180) {
16              throw new IllegalArgumentException("Longitud fuera de rango");
17          }
18
19          this.Latitud = latitud;
20          this.HemisferioLatitud = hemisferioLatitud;
21          this.HemisferioLongitud = hemisferioLongitud;
22          this.Longitud = longitud;
23      }
24
25      public double getLatitud() { no usages  👤 Irving Emanuel García Cedeño
26          return Latitud;
27      }
```

```
© CuerposCelestes.java  © PeriodoVisible.java  © UbicacionEspacial.java ×
3  public class UbicacionEspacial { 5 usages  👤 Irving Emanuel García Cedeño
29      public void setLatitud(double latitud) { no usages  👤 Irving Emanuel
30          Latitud = latitud;
31      }
32
33      public String getHemisferioLatitud() { no usages  👤 Irving Emanuel
34          return HemisferioLatitud;
35      }
36
37      public void setHemisferioLatitud(String hemisferioLatitud) {
38          HemisferioLatitud = hemisferioLatitud;
39      }
40
41      public double getLongitud() { no usages  👤 Irving Emanuel García Cedeño
42          return Longitud;
43      }
44
45      public void setLongitud(double longitud) { no usages  👤 Irving Emanuel
46          Longitud = longitud;
47      }
48
49      public String getHemisferioLongitud() { no usages  👤 Irving Emanuel
50          return HemisferioLongitud;
51      }
```



```

CuerposCelestes.java  PeriodoVisible.java  UbicaciónEspacial.java x
3      public class UbicaciónEspacial { 5 usages  Irving Emanuel García Cedeño
51      }
52
53      public void setHemisferioLongitud(String hemisferioLongitud) {
54      |   HemisferioLongitud = hemisferioLongitud;
55      }
56
57  @      public double calcularDesplazamiento(UbicacionEspacial otra){
58      |   double dx = this.Latitud - otra.Latitud;
59      |   double dy = this.Longitud - otra.Longitud;
60      |   return Math.sqrt(dx*dx + dy*dy);
61      }
62
63      @Override  Irving Emanuel García Cedeño
64  @      public String toString() {
65      |   return "ubicacionEspacial{" +
66      |       "longitud=" + Longitud +
67      |       ", latitud=" + Latitud +
68      |       '}';
69      }
70  }

```

Código clase periodoVisible:

```

CuerposCelestes.java  PeriodoVisible.java x  UbicaciónEspacial.java
1      package Observatorio;
2
3      import java.util.Arrays;
4      import java.util.List;
5
6      public class PeriodoVisible { 5 usages  Irving Emanuel García Cedeño
7      |   private String Periodo; 4 usages
8      |   private UbicaciónEspacial Ubicación; 4 usages
9
10     public PeriodoVisible(String periodo, UbicaciónEspacial ubicacion) { no usages  Irving Emanuel García
11     |   Periodo = periodo;
12     |   Ubicación = ubicacion;
13
14     |   List<String> meses = Arrays.asList(
15     |       "Enero", "Febrero", "Marzo", "Abril", "Mayo", "Junio",
16     |       "Julio", "Agosto", "Septiembre", "Octubre", "Noviembre", "Diciembre"
17     |   );
18
19     |   if (!meses.contains(periodo)) {
20     |       throw new IllegalArgumentException("El mes ingresado no es válido");
21     |   }
22
23     |   if (ubicacion == null) {
24     |       throw new NullPointerException("La ubicación es obligatoria para crear el periodo");
25     |   }
26     }

```

```

CuerposCelestes.java  PeriodoVisible.java  UbicacionEspacial.java
6      public class PeriodoVisible { 5 usages  Irving Emanuel Garcia Cedeño
28      public String getPeriodo() { no usages  Irving Emanuel Garcia Ced
29          return Periodo;
30      }
31
32      public void setPeriodo(String periodo) { no usages  Irving Em
33          Periodo = periodo;
34      }
35
36      public UbicacionEspacial getUbicacion() { no usages  Irving Er
37          return Ubicacion;
38      }
39
40      public void setUbicacion(UbicacionEspacial ubicacion) { no
41          Ubicacion = ubicacion;
42      }
43
44      @Override  Irving Emanuel Garcia Cedeño
45      public String toString() {
46          return "periodoVisible{" +
47              "periodo='" + Periodo + '\'' +
48              ", ubicacion=" + Ubicacion +
49              '}';
50      }

```

Prueba:

```

jshell> import Observatorio.*;

jshell> new UbicacionEspacial(200, "Norte", "Oeste", 50);
| Exception java.lang.IllegalArgumentException: Latitud fuera de rango
|   at UbicacionEspacial.<init> (UbicacionEspacial.java:12)
|   at (#17:1)

jshell> new UbicacionEspacial(20, "Norte", "Oeste", 300);
| Exception java.lang.IllegalArgumentException: Longitud fuera de rango
|   at UbicacionEspacial.<init> (UbicacionEspacial.java:16)
|   at (#18:1)

jshell>

jshell> UbicacionEspacial u1 = new UbicacionEspacial(20, "Norte", "Oeste", 50);
u1 ==> ubicacionEspacial{longitud=50.0, latitud=20.0}

jshell> new PeriodoVisible("MesX", u1);
| Exception java.lang.IllegalArgumentException: El mes ingresado no es válido
|   at PeriodoVisible.<init> (PeriodoVisible.java:20)
|   at (#20:1)

jshell> new PeriodoVisible("Enero", null);
| Exception java.lang.NullPointerException: La ubicación es obligatoria para crear el periodo
|   at PeriodoVisible.<init> (PeriodoVisible.java:24)
|   at (#21:1)

jshell> PeriodoVisible pv1 = new PeriodoVisible("Enero", u1);
pv1 ==> periodoVisible{periodo='Enero', ubicacion=ubicaci ... gitud=50.0, latitud=20.0}}

```

```
jshell> new CuerposCelestes("Sol", "Gas", 0, "KM", null);  
| Exception java.lang.NullPointerException: La lista de periodos no puede estar vacía  
|   at CuerposCelestes.<init> (CuerposCelestes.java:25)  
|   at (#23:1)
```

```
jshell>
```

```
jshell> new CuerposCelestes("Sol", "Gas", -10, "KM", new ArrayList<>());  
| Exception java.lang.IllegalArgumentException: Distancia inválida  
|   at CuerposCelestes.<init> (CuerposCelestes.java:21)  
|   at (#24:1)
```

```
jshell> ArrayList<PeriodoVisible> lista = new ArrayList<>();  
lista ==> []
```

```
jshell> lista.add(pv1);  
$26 ==> true
```

```
jshell> CuerposCelestes c1 = new CuerposCelestes("Sol", "Gas", 0, "KM", lista);  
c1 ==> cuerposCelestes{Nombre='Sol', Composicion='Gas', ... ra=0.0, UnidadTierra='KM'}
```

CONCLUSIONES:

Durante el desarrollo del programa, uno de los principales retos fue la implementación adecuada de las excepciones, algunas validaciones no se colocaron en el constructor.

Otra dificultad que se nos presentó fue la validación de los rangos numéricos, especialmente en los límites máximos y mínimos, en algunos casos se confundieron las condiciones (como usar operadores incorrectos como > en lugar de >=), lo que provocaba que aceptara valores que no debía.

También tuvimos complicaciones al validar porcentajes en el problema de frutas, ya que fue necesario calcular correctamente el rango permitido (entre 5% y 30% sobre el costo).

En conclusión, esta actividad no solo permitió practicar la creación de clases y métodos, sino también comprender la importancia del manejo de excepciones como mecanismo de control y seguridad en un sistema.

URI GIT:

<https://github.com/irvingemmanuel2003-a11y/actividad6.git>

<https://github.com/soysantiagorb/actividad6.git>